L'Ajax avec Axios et Symfony

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Récupération asynchrone des données **3**

II - Persistance des données avec Ajax **10**

A. Objectifs 10

III - Synthèse **17**

IV - Auto-évaluation **20**

A. Exercice : Développer un système de notifications asynchrones avec Axios et Symfony 20

B. Test 22

Tous droits réservés à STUDI - Reproduction interdite

I Récupération asynchrone des données

Durée : 2 h

Environnement de travail : visual Studio Code (ou tout autre éditeur de texte), PHP

Prérequis : PHP, Twig, JavaScript, DOM, Doctrine, bundle **Maker**, symfony, fixtures

Contexte

Le Web moderne évite au maximum les rechargements de page que peuvent occasionner les échanges entre le client et le serveur. Le but étant de rendre les sites web plus dynamiques, attractifs et rapides à consulter.

L'interaction entre l'utilisateur et le site a ainsi un impact direct sur le *Document Object Model* (ou DOM). Dans le cas de la soumission d'un formulaire, par exemple, l'impact est également propagé au serveur.

C'est donc dans ce cas de figure qu'on fera appel à l'AJAX. Il ne s'agit ni d'un langage, ni d'une technologie, mais bien d'un concept consistant à faire communiquer le client et le serveur de manière asynchrone, afin d'éviter les rechargements de page.

En d'autres termes il s'agit d'un ensemble de technologies qui permet la mise à jour du contenu d'une page Web d'une manière rapide et sans chargement complet de celle-ci. Ces technologies sont :

- Javascript et DOM pour le traitement,
- XML ou JSON pour l'extraction des données (le contenu à afficher),
- HTML et CSS pour la présentation.

Objectifs

- Découvrir l'asynchrone
- Découvrir l'AJAX
- Découvrir Axios au sein d'un projet Symfony
- Récupérer des données de manière asynchrone

Mise en situation

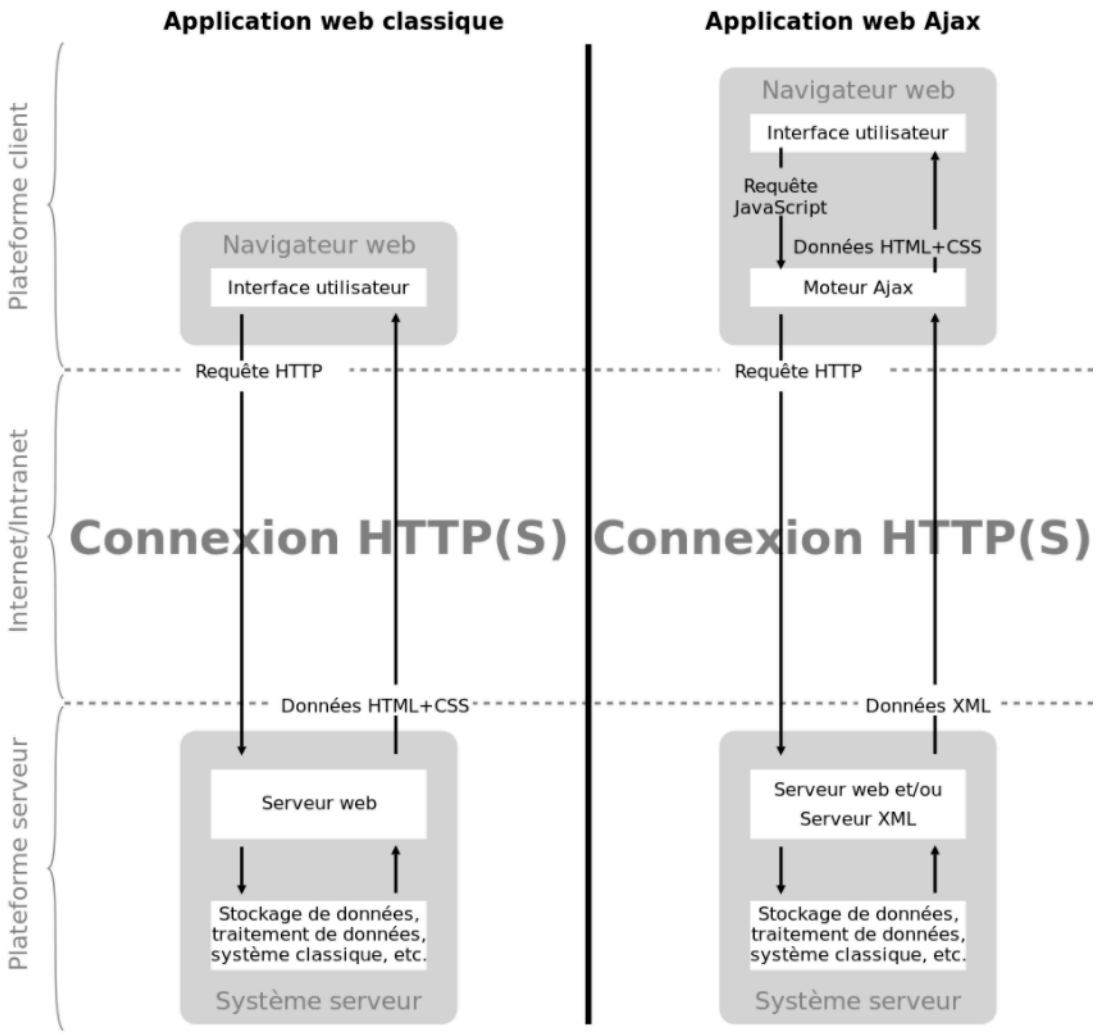
Nous l'avons évoqué précédemment, la récupération de données distantes sans rechargement de la page consultée rend l'expérience utilisateur plus agréable et la visite du site plus fluide.

Pour proposer une telle expérience dans nos sites et applications web, il faut entreprendre plusieurs étapes :

- L'utilisateur interagit avec un élément du DOM (un bouton, par exemple),
- Cet événement déclenche l'envoi de la requête vers le serveur HTTP,
- Le serveur traite à son tour la requête reçue,
- Le client reçoit la réponse ainsi traitée.

Tous droits réservés à STUDI - Reproduction interdite

Voici, pour illustrer le propos, un schéma issu de Wikipédia[1] :



L'AJAX (pour *Asynchronous JavaScript And XML*) est un concept utile au Web moderne. Il permet d'envoyer et de recevoir des données depuis une page web sans avoir à la recharger et ce, de manière **asynchrone**.

Pour comprendre l'asynchrone, nous allons pratiquer autour d'un exemple simple. Imaginons un site d'actualités permettant à ses utilisateurs connectés d'aimer ou non certains des articles publiés.

Bien entendu, nous avons déjà les cartes en main pour produire ce genre de site, mais nous rencontrerions des latences dans le chargement des données.

C'est pourquoi nous allons mettre en place certaines fonctionnalités de manière asynchrone afin d'offrir une expérience utilisateur plus agréable et fluide.

Avant de nous intéresser davantage à la partie concernant les articles publiés et la fonctionnalité permettant d'en aimer certains, nous allons mettre en plus un système asynchrone très simple, puisqu'il permet d'afficher ou masquer les adresses e-mail des utilisateurs du site.

1. https://upload.wikimedia.org/wikipedia/commons/thumb/2/2a/Ajax-modell-fr.svg/1024px-Ajax-modell-fr.svg.png

Tous droits réservés à STUDI - Reproduction interdite

Méthode

Pour mettre en place cette petite fonctionnalité, voici une entité `User` créée à l'aide du bundle **Maker**. Symfony Maker vous aide à créer des commandes vides, des contrôleurs, des classes de formulaire, des tests et plus encore. Un utilisateur est identifié par un e-mail et un mot de passe pour qu'il puisse se connecter au site web.

Pour formater correctement les données avant de les envoyer au client JavaScript, on a ajouté une méthode très simple à la classe `User`.

```
public function toArray() {
        return [
            "id" => $this->getId(),
            "email" => $this->getEmail(),
        ];
    }
```

Méthode

Comme son nom l'indique, la méthode `toArray()` va transformer l'objet PHP depuis lequel elle est appelée en tableau associatif contenant toutes les informations que l'on a besoin de faire passer au client : l'`id` et l'adresse e-mail de l'utilisateur.

Nous avons également mis en place un minimum de sécurité à l'aide d'un formulaire de connexion basique.

Plus intéressant maintenant, voyons comment le contrôleur `UserController` a été construit.

```
<?php

namespace App\Controller;

use App\Repository\UserRepository;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Attribute\Route;

class UserController extends AbstractController {
    #[Route('/users', name: 'read_users')]
    public function read(UserRepository $userRepository) {
        $users = $userRepository->findAll();
        $arrayOfUsers = [];
        foreach ($users as $user) {
            $arrayOfUsers[] = $user->toArray();
        }
        return $this->json($arrayOfUsers);
    }
}
```

C'est à ce moment que l'on entre dans le cœur du sujet, puisque l'on va configurer les données reçues afin que notre client JavaScript les reçoive au format JSON.

Pour ce faire, nous récupérons d'abord tous les utilisateurs en base de données (vous pouvez les générer à l'aide de fixtures, cf *Doctrine Fixture*), avant de les stocker un à un dans un tableau, non pas sous la forme d'un objet PHP, mais d'un tableau associatif généré à l'aide de la méthode `toArray()`, préalablement ajoutée à l'entité `User`.

Tous droits réservés à STUDI - Reproduction interdite

Une fois tous les utilisateurs stockés dans le tableau, nous le passons en argument de la méthode `json()`, issue de la classe `AbstractController` et permettant de renvoyer une réponse au format JSON.

De cette manière, nous pourrons réaliser un appel asynchrone depuis notre client sur la route `/users` pour récupérer ces données.

Intéressons-nous maintenant à l'aspect front-end de notre application. Pour cela, nous avons créé un contrôleur `PostController` basique, qui nous permettra d'afficher les différents articles publiés directement depuis la route racine de l'application. En l'occurrence, le template Twig affichera également un bouton « ***Afficher les utilisateurs*** » pour afficher la liste des adresses e-mail des utilisateurs stockés en base de données de manière asynchrone.

Voyons à quoi ressemble le contrôleur `PostController`.

```php
<?php

namespace App\Controller;

use App\Repository\PostRepository;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\Routing\Attribute\Route;

class PostController extends AbstractController {
    #[Route('/', name: 'posts')]
    public function index(PostRepository $postRepository) {
        return $this->render("post/index.html.twig", [
           "posts" => $postRepository->findAll()
        ]);
    }
}

?>
```

De manière tout à fait basique, ce contrôleur se contente de renvoyer tous les articles stockés en base de données au template `index.html.twig`, situé dans le dossier `./templates/post`. Comme convenu, la route racine de notre application lui est associée.

Intéressons-nous enfin au fichier `index.html.twig` contenant la logique asynchrone de la fonctionnalité que nous souhaitons implémenter.

```twig
{% extends 'base.html.twig' %}

{% block body %}
    <h2>Mes publications</h2>
    <a href="/users" class="btn btn-primary js-users">Afficher les utilisateurs</a>
    <div class="users"></div>
    <div class="row">
        {% for post in posts %}
            <div class="col-3 mb-3 card" style="width: 18rem;">
                <div class="card-body">
                    <h5 class="card-title">{{ post.title }}</h5>
                    <p class="card-text">{{ post.content }}</p>
                </div>
            </div>
        {% else %}
            <b>Aucune publication</b>
        {% endfor %}
    </div>
{% endblock %}

{% block javascripts %}
    <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
    <script>
```


Tous droits réservés à STUDI - Reproduction interdite

```
        window.addEventListener("load", function(event) {
            document.querySelector("a.js-users").addEventListener("click", getUsers);
        });

        function getUsers (event) {
            event.preventDefault();
            axios.get(this.href).then(response => {
                const users = document.querySelector("div.users");
                if(this.classList.contains("btn-primary")) {
                    response.data.forEach(user => {
                        const node = document.createElement("li");
                        node.textContent = user.email;
                        users.appendChild(node);
                    });
                    this.classList.replace("btn-primary", "btn-danger");
                    this.textContent = "Masquer les utilisateurs";
                } else {
                    users.innerHTML = "";
                    this.classList.replace("btn-danger", "btn-primary");
                    this.textContent = "Afficher les utilisateurs";
                }
            }).catch(error => {
                console.error(error);
                window.alert("Une erreur s'est produite.");
            });
        }
    </script>
{% endblock %}
```

Je vous l'accorde, le fichier est bien rempli. Prenons le temps de le décortiquer ensemble.

Comme dans tout template Twig, nous étendons d'abord le fichier `base.html.twig` afin d'avoir accès aux différents blocs de structure. Nous ouvrons justement le bloc `body` afin d'y placer le corps de notre page web.

Un titre, puis des cartes stylisées à l'aide de Bootstrap pour afficher les articles stockés en base de données, envoyés via le contrôleur `PostController`. Dans le cas où il n'y a pas d'article, on affiche un message simple, mais efficace, en gras : « ***Aucune publication*** ».

Voilà pour la base de ce fichier. Il s'agit là d'éléments que vous connaissez déjà. Intéressons-nous maintenant à de l'inédit !

Souvenez-vous, nous avons dit que nous souhaitions afficher les adresses e-mail des utilisateurs stockés en base de données de manière asynchrone, et ce d'un simple clic sur un bouton. Nous avons justement mis en place un bouton « ***Afficher les utilisateurs*** » pointant vers l'URL `/users` précédemment configurée à travers le contrôleur `UserController`.

Remarquez également que nous lui avons donné une classe Bootstrap afin de le styliser, mais également une classe `js-users` afin de traiter l'événement lié au clic sur ce bouton.

Pour ce faire, il nous faut écrire un peu de JavaScript, ce qui nous mène directement au second bloc du fichier, le bloc `javascripts`.

La première chose à entreprendre pour faire de l'AJAX au sein de notre template Twig va être de charger Axios. Il s'agit d'un client HTTP basé sur les promesses JavaScript. Pour rappel, l'objet `Promise` est utilisé pour réaliser des traitements asynchrones.

Pour charger cette technologie, nous utilisons simplement son CDN (pour *Content Delivery Network* ou Réseau de Diffusion de Contenu) référencé sur le GitHub officiel d'Axios, disponible à cette adresse : axios/axios[2].

Ce faisant, nous avons désormais accès à l'objet JavaScript `axios` permettant de réaliser des requêtes de manière asynchrone.

---

2. https://github.com/axios/axios

Tous droits réservés à STUDI - Reproduction interdite

Avant cela, nous allons vouloir que la page soit complètement chargée avant d'ajouter un écouteur d'événement sur le bouton « ***Afficher les utilisateurs*** ». Pour ce faire, nous utilisons à deux reprises la méthode `addEventListener()` afin de gérer le chargement de la page, puis le bouton qui nous intéresse que nous récupérons très simplement grâce à la méthode `querySelector()` associée au sélecteur CSS `a.js-users`, que nous avons paramétré précédemment, passé en argument.

Remarquez l'appel de la fonction `getUsers()` lorsque l'utilisateur clique sur le bouton. C'est au sein de celle-ci que nous allons réaliser notre premier traitement asynchrone.

Comme toute fonction appelée depuis la méthode `addEventListener()`, elle reçoit l'événement produit en argument.

Au sein de la fonction `getUsers()`, on utilise directement cet objet afin d'éviter la redirection vers la route `/users` que nous avons programmée à travers l'attribut `href` associé au bouton « ***Afficher les utilisateurs*** ». En effet, la méthode `preventDefault()` associée à un événement JavaScript permet d'éviter tout autre traitement.

Il est maintenant question d'utiliser l'objet `axios` afin de réaliser les traitements associés à la route `/users` de manière asynchrone. Pour ce faire, Axios propose deux méthodes :

- La méthode `get()` permet de requêter une URL, passée en argument, avec des paramètres éventuels .
- La méthode `post()` permet de requêter une URL, passée en argument, avec des informations complémentaires à placer dans le corps de la requête.

En l'occurrence, nous souhaitons uniquement requêter l'URL `/users`, qui a été passée en valeur de l'attribut `href` du bouton cliqué. Ainsi, le mot-clé `this` nous permet de récupérer cet élément HTML, duquel nous récupérons la valeur associée à l'attribut `href`.

En appelant la méthode `get()`, issue de l'objet `axios`, nous créons une promesse JavaScript. Nous avons ainsi accès à la méthode `then()` permettant de réaliser des traitements une fois la réponse HTTP reçue.

Cet objet est d'ailleurs passé en argument de la fonction anonyme que reçoit la méthode `then()` afin de la traiter.

Souvenez-vous, nous avons fait en sorte que le contrôleur associé à la route `/users` se charge de renvoyer tous les utilisateurs stockés en base de données au format JSON.

Ces données sont désormais accessibles directement depuis l'objet `response`. Pour en avoir le cœur net, vous pouvez afficher l'objet dans la console de votre navigateur web via l'expression `console.log(response)`.



Nous découvrons ici plusieurs informations intéressantes. Tout d'abord, le code de statut 200 renvoyé par la requête : tout s'est bien passé. Nous retrouvons également l'URL requêtée, ainsi que la méthode HTTP utilisée, le type de données renvoyées, l'en-tête de la réponse, etc.

Ce qui nous intéresse se situe dans la propriété `data` de l'objet `response`. Il s'agit bel et bien du tableau de données JSON que nous avons renvoyé depuis le contrôleur `UserController`.

Il ne nous reste plus qu'à parcourir ce tableau de données afin de les traiter avec le JavaScript nécessaire.

Tous les traitements réalisés ici ont un impact direct sur le DOM de la page. Nous récupérons d'abord la `div` vide associée à la classe CSS `users`, préalablement créée, puis nous vérifions que le bouton sur lequel nous avons cliqué est associé à la classe `btn-primary`.

Si c'est le cas, nous parcourons le tableau de données situé dans la propriété `data` de l'objet `response`, afin d'insérer chacune des adresses e-mail des utilisateurs en éléments d'une liste HTML non ordonnée.

Tous droits réservés à STUDI - Reproduction interdite

Enfin, nous remplaçons le texte du bouton « ***Afficher les utilisateurs*** », par « ***Masquer les utilisateurs*** ». Nous en profitons également pour remplacer la classe `btn-primary` par `btn-danger` pour faire passer le bouton Bootstrap du bleu au rouge.

Dans le cas où la classe `btn-danger` est déjà attribuée au bouton cliqué, il nous faut retirer les enfants de la div `users` et revenir vers le texte et le style précédents du bouton.

Remarque

Quand nous parlons d'asynchrone et de `Promise` JavaScript, nous ne sommes pas à l'abri de rencontrer une erreur liée au serveur, au manque de connexion, ou directement au client.

Pour gérer correctement ce genre de problème et ainsi prévenir l'utilisateur du problème rencontré, nous avons accès à la méthode `catch()`, prenant en argument une fonction anonyme recevant l'erreur rencontrée en paramètre. Nous pouvons ensuite traiter l'erreur comme nous le souhaitons. En l'occurrence, nous choisissons de l'afficher en rouge en console et de prévenir l'utilisateur à l'aide d'une simple modale d'alerte.



Ici, nous avons volontairement choisi de faire pointer la requête asynchrone vers une URL qui n'existe pas dans notre application. L'erreur nous signale que l'on est face à un code de statut 404. On pourrait ainsi imaginer un traitement simple permettant de signaler à l'utilisateur qu'il se trouve sur une URL inexistante, par exemple.

À retenir

Syntaxe

Dans cette partie du cours, plusieurs éléments méritent une attention toute particulière de votre part.

Cette information est disponible depuis le GitHub officiel d'Axios[3], mais il ne fait pas de mal de rappeler son CDN : `<script src="https://unpkg.com/axios/dist/axios.min.js"></script>`.

Il suffit de l'intégrer dans un un template Twig pour pouvoir utiliser l'objet JavaScript `axios` permettant de réaliser des traitements de manière asynchrone, notamment grâce à sa méthode `get()` qui, en lui passant une URL en argument, permet de la requêter à l'aide d'une `Promise` JavaScript.

L'asynchrone sous-entend que nous ne savons pas nécessairement quand nous allons recevoir les données souhaitées. Aussi, la méthode `then()` va nous permettre de réaliser les traitements qui ne doivent être faits sur la réponse HTTP qu'une fois reçue.

Cette réponse HTTP est justement contenue dans un objet `response` passé en argument d'une fonction anonyme que prend la méthode `then()`. À vous de traiter les éléments contenus dans la réponse pour combler vos attentes. Retenez que vous pouvez afficher cet objet en console de votre navigateur web à l'aide de l'expression `console.log(response)`.

Enfin, la méthode `catch()`, à faire suivre du bloc `then()`, permet de stopper l'exécution du script pour traiter correctement les erreurs rencontrées. Il est ainsi facile de prévenir l'utilisateur de ce qui ne va pas grâce à l'objet JavaScript `error`, transmis en argument de la fonction anonyme reçue par la méthode `catch()`.

3. https://github.com/axios/axios

Tous droits réservés à STUDI - Reproduction interdite

II Persistance des données avec Ajax

A. Objectifs

- Découvrir AJAX
- Découvrir Axios
- Persister des données de manière asynchrone

Mise en situation

Nous savons à présent faire communiquer de manière asynchrone le client et le serveur. Pour aller plus loin, il serait intéressant de créer des données directement depuis le client HTTP. Pour ce faire, il est possible d'incorporer, dans le corps de la requête HTTP, des données qui seront utilisées par le serveur.

Les verbes HTTP utilisés seront `POST`, `PUT`, `PATCH`, `DELETE`, etc.

Après l'envoi de la requête, certains traitements devront être mis en place du côté du serveur afin de renvoyer une réponse respectant plusieurs normes :

- Le format (du JSON, principalement)
- Le code de statut de la réponse HTTP

Pour comprendre un peu plus l'ensemble des possibilités qu'offre l'AJAX, nous allons permettre aux utilisateurs connectés à notre application d'aimer ou d'arrêter d'aimer un article.

Vous l'aurez compris, il s'agira de créer dynamiquement des données en réponse à l'événement associé à un clic sur un bouton « ***J'aime*** ».

Méthode

Pour mettre en place un tel système, nous avons besoin de créer une nouvelle entité `PostLike`. Ainsi, un objet issu de cette entité aura un article et un utilisateur associés. Un article peut être aimé par un ou plusieurs utilisateurs, et un utilisateur peut aimer un ou plusieurs articles.

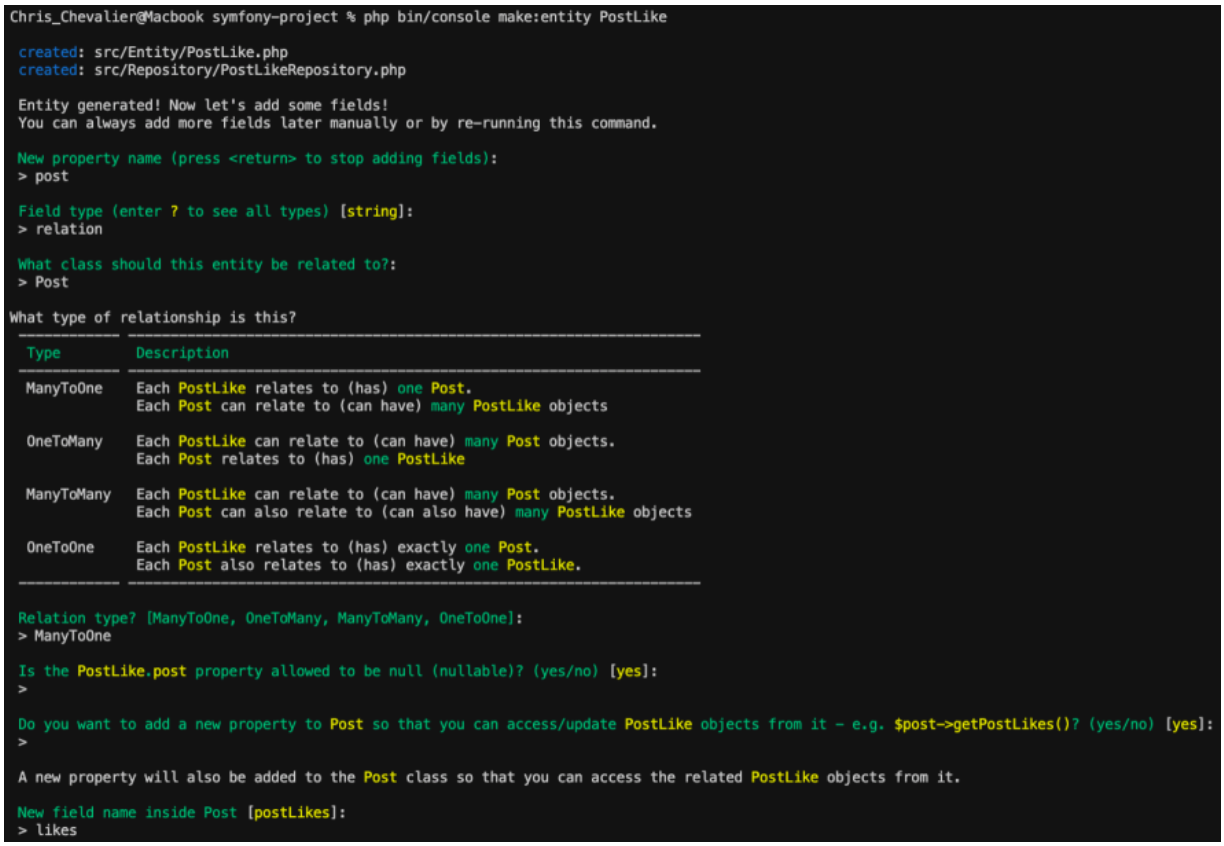
Utilisons le bundle **Maker** pour créer cette entité simplement. Nous saisissons donc la commande `$ php bin/console make:entity PostLike`.

Nous allons paramétrer seulement 2 propriétés pour cette entité :

- L'article aimé (`post`) de type `relation ManyToOne` et associé à l'entité `Post` contenant ainsi une nouvelle propriété `likes`.
- L'utilisateur qui a aimé l'article (`user`) de type `relation ManyToOne` et associé à l'entité `User` contenant ainsi une nouvelle propriété `likes`.

Tous droits réservés à STUDI - Reproduction interdite

Voyons ce que donne notre terminal pour la configuration de la propriété `post` :



Une fois l'entité `PostLike` correctement configurée, mettons notre base de données à jour à l'aide des commandes `$ php bin/console make:migrations`, puis `$ php bin/console doctrine:migrations:migrate`.

Pour gérer correctement notre système de *likes*, nous allons ajouter une méthode `isLikedByUser()` à l'entité `Post` permettant de savoir si l'article testé a été aimé ou non par l'utilisateur passé en paramètre.

```
1 public function isLikedByUser(User $user): bool {
2         foreach ($this->likes as $like) {
3             if($like->getUser() === $user) {
4                 return true;
5             }
6         }
7         return false;
8     }
```

La signature de cette méthode nous permet de savoir qu'elle prend un utilisateur en argument et retourne un booléen. Elle se contente en réalité de parcourir le tableau associé à la propriété `$likes` de l'article courant, et vérifie à chaque itération de boucle si l'utilisateur passé en paramètre est le même que celui du *like* parcouru. Si c'est le cas, elle renvoie la valeur `true`. Si elle a parcouru chacun des *likes* associés à l'article sans jamais être tombé sur un donné par l'utilisateur passé en argument, elle renvoie la valeur `false`.

Nous pouvons ainsi savoir très simplement si un utilisateur donné à aimer ou non un article donné.

Cette méthode va nous servir dans la mise en place de notre interface utilisateur, mais aussi dans la méthode `like()`, issue de la classe `PostController`, permettant de faire aimer un article à un utilisateur.

Mettons cette méthode en place sans plus attendre !

Tous droits réservés à STUDI - Reproduction interdite

```php
#[Route('/post/{id}/like', name: 'post_like')]
    public function like(Post $post, EntityManagerInterface $em, PostLikeRepository $likeRepo): Response {
        $user = $this->getUser();
        if(!$user) {
            return $this->json(["code" => 403, "message" => "Unauthorized"], 403);
        }
        if($post->isLikedByUser($user)) {
            $like = $likeRepo->findOneBy([
                "post" => $post,
                "user" => $user
            ]);
            $em->remove($like);
            $em->flush();
            return $this->json([
                "code" => 200,
                "message" => "Like bien supprimé",
                "likes" => $likeRepo->count(["post" => $post])
            ], 200);
        }

        $like = (new PostLike())
            ->setPost($post)
            ->setUser($user);
        $em->persist($like);
        $em->flush();

        return $this->json([
            "code" => 200,
            "message" => "Like bien ajouté",
            "likes" => $likeRepo->count(["post" => $post])
        ], 200);
    }
```

Nous découvrons dans sa signature que cette méthode a besoin de plusieurs objets PHP pour fonctionner correctement. Ainsi, elle reçoit l'article à aimer (`$post`), une instance de la classe `EntityManagerInterface` (`$em`) pour synchroniser les objets modifiés ou créés avec la base de données, et une instance de la classe `PostLikeRepository` (`$likeRepo`) pour retrouver un *like* enregistré en base.

Comme nous l'avons fait dans la partie précédente de ce cours, nous souhaitons ici renvoyer un contenu au format JSON. En l'occurrence, il serait intéressant de renvoyer, en plus des informations contenues dans la réponse HTTP, un code de statut, ainsi qu'un message associé au traitement réalisé. Si tout se passe bien, il serait également bon d'ajouter à ces informations le nombre de *likes* associés à l'article après l'ajout de celui que l'on crée, ou de sa suppression.

Dans un premier temps, assurons-nous que l'utilisateur est connecté. Dans un contrôleur Symfony, on récupère l'utilisateur connecté avec la méthode `getUser()` issue de la classe `AbstractController`, qui étend chacun de nos contrôleurs.

Ainsi, si la méthode `getUser()` renvoie une valeur nulle, nous savons que l'utilisateur ne peut pas aimer un article. Nous renvoyons donc une réponse au format JSON avec un code de statut « *403 – Non autorisé* » et un message plus explicite.

Remarquez que la méthode `json()`, issue encore une fois de la classe `AbstractController`, peut recevoir un second argument en plus des données que nous souhaitons transmettre à la réponse HTTP. Ce second argument représente simplement un entier relatif au code de statut associé à un tel traitement.

Sur les réseaux sociaux, quand un utilisateur clique sur un bouton « ***J'aime*** », cela n'ajoute pas forcément un nouveau *like* en base de données. Eh oui, ça l'ajoute si l'utilisateur n'a pas déjà aimé le contenu, et le retire sinon. Traitons en premier lieu ce cas de figure, à l'aide de la méthode `isLikedByUser()` précédemment mise en place.

Tous droits réservés à STUDI - Reproduction interdite

L'appel de cette méthode depuis l'instance de `Post` permet de savoir si un article a été aimé par l'utilisateur passé en argument. Ainsi, si c'est le cas, on récupère d'abord l'instance de `PostLike` via la méthode `findOneBy()`, issue de la classe `PostLikeRepository`.

Nous savons donc que nous souhaitons récupérer le *like* associé à l'article passé en paramètre et à l'utilisateur connecté.

Nous utilisons ensuite l'instance `$em` de l'`EntityManagerInterface` pour supprimer le *like* de la base de données, avant de synchroniser le tout via la méthode `flush()`.

Nous pouvons enfin retourner une réponse au format JSON contenant un message positif, un code de statut « *200 – OK* » et le nouveau nombre de *likes* associés à l'article.

Si nous ne rentrons pas dans la condition précédemment détaillée, cela signifie que l'article n'a pas été aimé par l'utilisateur connecté. Nous créons alors une nouvelle instance de la classe `PostLike` en l'associant à l'article courant et à l'utilisateur connecté, avant de persister le tout en base de données.

Nous renvoyons ici aussi une réponse au format JSON contenant un message positif, un code de statut 200 et le nouveau nombre de *likes* pour cet article.

Enfin, remarquez que ces traitements côté serveur sont associés à la route `/post/{id}/like`. Nous allons nous servir de cette route pour réaliser nos traitements asynchrones directement depuis un template Twig :

```
{% extends 'base.html.twig' %}

{% block body %}
    <h2>Mes publications</h2>
    {% if app.user %}
        <a href="/logout" class="btn btn-danger">Déconnexion</a>
    {% else %}
         <a href="/login" class="btn btn-success">Connexion</a>
    {% endif %}
    <div class="row">
        {% for post in posts %}
            <div class="col-3 mb-3 card" style="width: 18rem;">
                <div class="card-body">
                    <h5 class="card-title">{{ post.title }}</h5>
                    <p class="card-text">{{ post.content }}</p>
                    <a href="/post/{{post.id}}/like" class="btn btn-link js-like">
                    {% if app.user and post.isLikedByUser(app.user) %}
                        <i class="fas fa-thumbs-up"></i>
                    {% else %}
                        <i class="far fa-thumbs-up"></i>
                    {% endif %}
                        <span class="js-likes">{{ post.likes|length }}</span>
                        <span>J'aime</span>
                    </a>

                </div>
            </div>
        {% else %}
            <b>Aucune publication</b>
        {% endfor %}
    </div>
{% endblock %}

{% block javascripts %}
    <script src="https://cdn.jsdelivr.net/npm/axios@1.7.9/dist/axios.min.js">
</script>
    <script>
        window.addEventListener("load", function(event) {
            document.querySelectorAll("a.js-like").forEach(link => {
```

```
                link.addEventListener("click", like)
            });
        });

        function like (event) {
            event.preventDefault();
            const spanCount = this.querySelector("span.js-likes");
            const likeIcon = this.querySelector("i.fa-thumbs-up");

            axios.post(this.href).then(response => {
                spanCount.textContent = response.data.likes;
                if(likeIcon.classList.contains("fas")) {
                    likeIcon.classList.replace("fas", "far");
                } else {
                    likeIcon.classList.replace("far", "fas");
                }
            }).catch(error => {
                if(error.response.status === 403) {
                    window.alert("Vous ne pouvez pas aimer un article sans être
connecté.");
                } else {
                    window.alert("Une erreur s'est produite.");
                }
            });
        }
    </script>
{% endblock %}
```

Remarquez que le fichier Twig ressemble sous bien des aspects à celui que nous avons découvert dans la première partie de ce cours. Seuls certains éléments d'interface bienvenus ont été ajoutés.

Nous avons d'abord ajouté un bouton de connexion ou de déconnexion associé aux routes paramétrées dans la configuration de la sécurité Symfony (sur laquelle on ne reviendra pas dans ce cours). Simplement, le bouton « ***Déconnexion*** » apparaît si l'utilisateur est connecté, et inversement. En Twig, nous récupérons l'utilisateur connecté en passant par la variable superglobale app. Cela donne donc `app.user`.

Plus bas dans le code, nous découvrons un nouveau bouton Bootstrap associé à la route qui nous intéresse : `/post/{id}/like`. Malheureusement, en tant que telle, elle n'offre pas un grand dynamisme à notre application.

C'est pour cette raison que nous avons associé cet élément HTML à la classe CSS `js-like`. Nous y reviendrons.

Tous droits réservés à STUDI - Reproduction interdite

Par la suite, nous découvrons une simple vérification permettant de savoir si l'article a été aimé ou non par l'utilisateur connecté. Si c'est le cas, nous utilisons l'icône Font Awesome[4] de pouce en l'air `solid`. Dans le cas contraire, nous utilisons l'icône `regular`. Voici la différence entre ces deux icônes :



Nous affichons enfin le label du bouton. Nous faisons apparaître d'abord le nombre de *likes* de l'article, suivi du texte « ***J'aime*** ». Pour récupérer le nombre de *likes* sur un article, il suffit d'utiliser le filtre Twig `length` sur la propriété `likes` associé au `post` actuellement parcouru.

Intéressons-nous maintenant à la partie de cette page web écrite en JavaScript.

Comme dans la partie précédente, nous nous assurons d'abord que la page a terminé de charger son contenu avant de boucler sur chacun des éléments a du DOM associés à la classe CSS `js-like`, que nous avons préalablement paramétrée.

Nous bouclons ainsi sur chacun des boutons « ***J'aime*** » afin d'ajouter un écouteur d'événement sur chacun d'entre eux. De cette manière, lorsque l'utilisateur clique sur ce bouton, la fonction `like()`, située plus bas dans le code est exécutée.

Nous évitons toute redirection vers l'URL passée en attribut `href` de l'élément HTML cliqué grâce à la méthode `preventDefault()` relative à l'événement du clic, passé en paramètre.

Par la suite, nous récupérons et stockons dans deux variables les éléments du DOM correspondant au nombre de *likes* et à l'icône affichée afin de pouvoir les modifier une fois la réponse HTTP reçue.

Pour la recevoir, il nous faut bien entendu requêter l'URL, avec cette fois la méthode `post()`, puisque nous cherchons ici à créer de nouvelles données. Même s'il n'y a pas d'intérêt particulier à le faire dans cet exemple, sachez qu'il est possible de faire passer un objet JavaScript dans le corps de la requête.

4. https://fontawesome.com/

Tous droits réservés à STUDI - Reproduction interdite

Le GitHub officiel d'Axios nous dévoile un exemple parlant d'une telle utilisation :

```
axios.post('/user', {
    firstName: 'Fred',
    lastName: 'Flintstone'
  })
  .then(function (response) {
    console.log(response);
  })
  .catch(function (error) {
    console.log(error);
  });
```

En quelques mots, nous requêtons ici la route `/user` en passant les prénom et nom d'un utilisateur dans le corps de la requête, afin de les traiter côté serveur, et avant de récupérer la réponse HTTP, toujours de manière asynchrone.

Dans notre exemple, nous n'avons pas besoin d'envoyer quoi que ce soit au serveur, puisqu'il a déjà tout ce dont il a besoin en paramètres de l'URL pour traiter la requête.

Souvenez-vous, la réponse reçue de manière asynchrone doit avoir le format JSON que nous avons précédemment paramétré côté serveur, dans la méthode `like()` de la classe `PostController`.

Si tout se passe bien, nous n'entrons pas dans le bloc `catch()`, mais bien dans le bloc `then()` pour traiter la réponse reçue et ainsi mettre à jour le contenu textuel associé à l'élément HTML contenant le nombre de *likes* avec la valeur de la propriété `likes`, récupérée depuis les données JSON de la réponse.

Nous réalisons ensuite un simple traitement permettant de mettre à jour l'aspect de l'icône de pouce en l'air, en `solid` s'il était `regular`, et inversement.

Nous traitons enfin correctement les erreurs éventuelles reçues. Dans le cas où le code de statut est « *403 – Non autorisé* », nous le signalons à l'utilisateur au travers d'une modale d'alerte. Dans tous les autres cas de figure, nous lui signalons simplement qu'une erreur est survenue.

À retenir Syntaxe

Vous l'avez remarqué, la méthode `post()`, issue de l'objet `axios`, fonctionne de la même manière que `get(` . À ceci près qu'il est possible ici de lui faire passer un second argument représentant du contenu JSON. Ce contenu sera accessible depuis le contrôleur Symfony, associé à la route requêtée grâce à la méthode `get()`, issue d'un objet `request`, lui-même issu de la classe `Symfony\Component\HttpFoundation\Request`. En d'autres termes, nous pourrions récupérer le prénom d'un utilisateur passé dans le corps de la requête en écrivant `$request->request->get(« firstname »)`.

Tous droits réservés à STUDI - Reproduction interdite

III Synthèse

A. Flashcards

Ajax

Concept de communication asynchrone client-serveur sans rechargement de page.

Axios

Client HTTP JavaScript basé sur les promesses pour requêtes asynchrones.

Méthode get() Axios

Requête HTTP de lecture asynchrone vers une URL avec paramètres optionnels.

Méthode post() Axios

Requête HTTP d'écriture asynchrone avec corps JSON vers une URL.

Méthode then()

Traite la réponse HTTP reçue après résolution d'une promesse JavaScript.

Méthode catch()

Gère les erreurs survenues lors d'une requête asynchrone.

Méthode json() Symfony

Renvoie une réponse HTTP au format JSON avec code de statut.

EntityManagerInterface

Interface Symfony pour persister, supprimer et synchroniser entités en base.

Méthode getUser() Symfony

Récupère l'utilisateur actuellement connecté dans un contrôleur Symfony.

Promesse JavaScript

Objet représentant l'achèvement ou l'échec futur d'une opération asynchrone.

Code statut HTTP 200

Indique que la requête a réussi et le traitement s'est déroulé correctement.

Code statut HTTP 403

Indique que l'accès à la ressource est interdit pour l'utilisateur.

Méthode preventDefault()

Empêche le comportement par défaut d'un événement JavaScript.

CDN Axios

Réseau de diffusion de contenu permettant de charger Axios via URL externe.

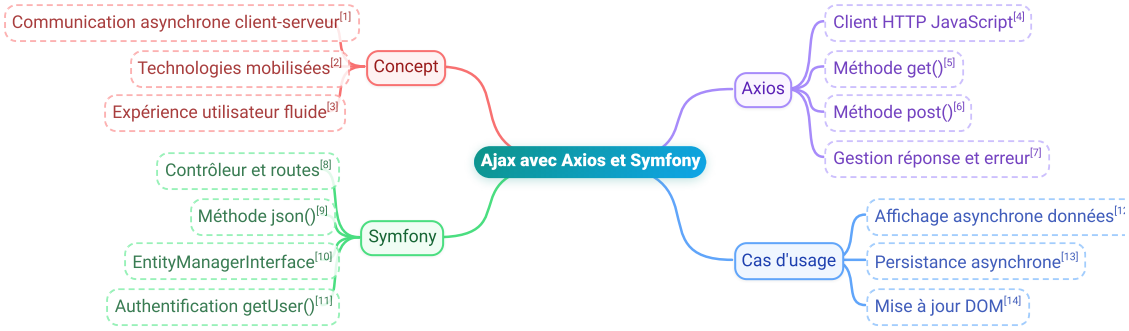
La théorie en bref

Fondamental

L'Ajax (Asynchronous JavaScript And XML) est un concept permettant de faire communiquer le **client** et le **serveur** de manière **asynchrone**, sans rechargement de page. Il repose sur **JavaScript**, le **DOM**, **JSON** ou XML pour les données, et **HTML/CSS** pour la présentation. **Axios** est un client HTTP basé sur les **promesses JavaScript**, utilisable via CDN, qui expose les méthodes **get()** et **post()** pour requêter des URL. La méthode **then()** traite la réponse HTTP une fois reçue, tandis que **catch()** gère les erreurs. Côté Symfony, la méthode **json()** de

Tous droits réservés à STUDI - Reproduction interdite

l'AbstractController renvoie une réponse au format JSON avec un code de statut HTTP. L'utilisateur connecté est récupérable via **getUser()**, et l'**EntityManagerInterface** permet de persister ou supprimer des entités en base de données.



1. Échanges sans rechargement de page.
2. JavaScript, DOM, JSON/XML, HTML, CSS.
3. Réactivité et dynamisme accrus.
4. Basé sur les promesses.
5. Requête de lecture avec paramètres URL.
6. Requête d'écriture avec corps JSON.
7. then() pour succès, catch() pour erreurs.
8. Exposition d'URL pour requêtes Ajax.
9. Renvoie réponse JSON avec code statut.
10. Persist, remove, flush pour base de données.
11. Récupération utilisateur connecté.
12. Liste utilisateurs sans rechargement.
13. Ajout/suppression like ou bookmark.
14. Modification icônes, compteurs, classes CSS.

Ajax avec Axios et Symfony

Les conseils pro

Conseil

Comment structurer un contrôleur Symfony pour exposer des données au format JSON de manière sécurisée ?

- Vérifier systématiquement l'authentification de l'utilisateur via getUser() avant tout traitement sensible.
- Utiliser la méthode json() d'AbstractController pour formater la réponse avec un code de statut HTTP explicite.
- Créer une méthode toArray() dans chaque entité pour contrôler précisément les données exposées au client.
- Renvoyer des messages d'erreur clairs et des codes de statut normalisés (401, 403, 500) pour faciliter le débogage côté client.

Comment implémenter une requête asynchrone Axios robuste dans un template Twig ?

Tous droits réservés à STUDI - Reproduction interdite

- Charger Axios via CDN dans le bloc javascripts du template Twig pour garantir sa disponibilité.
- Attendre le chargement complet du DOM avec l'événement load avant d'ajouter les écouteurs d'événements.
- Utiliser preventDefault() sur l'événement de clic pour éviter toute redirection non souhaitée.
- Encadrer systématiquement les appels Axios avec then() pour le succès et catch() pour les erreurs, en affichant des messages utilisateur explicites.

Comment choisir entre axios.get() et axios.post() pour une fonctionnalité asynchrone ?

- Utiliser get() pour récupérer des données sans modifier l'état du serveur (lecture seule).
- Privilégier post() pour créer, modifier ou supprimer des ressources côté serveur (écriture).
- Passer les paramètres dans l'URL pour get(), et dans le corps de la requête JSON pour post().
- Documenter le choix dans le code pour faciliter la maintenance et la compréhension par l'équipe.

Comment gérer efficacement les erreurs asynchrones côté client avec Axios ?

- Inspecter la propriété error.response.status dans le bloc catch() pour identifier le type d'erreur HTTP.
- Afficher des messages d'erreur contextualisés selon le code de statut (403 pour non autorisé, 500 pour erreur serveur).
- Logger l'erreur complète en console avec console.error() pour faciliter le diagnostic technique.
- Prévoir un message générique pour les erreurs non anticipées afin de ne jamais laisser l'utilisateur sans retour.

Comment maintenir la cohérence entre le DOM et l'état serveur après une requête asynchrone ?

- Mettre à jour les éléments du DOM uniquement après réception de la réponse HTTP dans le bloc then().
- Utiliser les données renvoyées par le serveur (compteurs, états) pour synchroniser l'affichage client.
- Manipuler les classes CSS et le contenu textuel des éléments HTML pour refléter visuellement les changements d'état.
- Tester systématiquement les scénarios d'échec pour vérifier que le DOM reste cohérent même en cas d'erreur serveur.

Tous droits réservés à STUDI - Reproduction interdite

IV Auto-évaluation

A. Exercice : Développer un système de notifications asynchrones avec Axios et Symfony

Cas pratique

Vous venez d'intégrer l'équipe de développement web de **NovaBlog**, une plateforme de publication d'articles destinée aux professionnels du secteur tech. L'application est construite avec **Symfony** et utilise **Twig** pour le rendu côté client.

La responsable technique, Samira Khelifi, vous confie une mission : enrichir l'expérience utilisateur de la plateforme en ajoutant des fonctionnalités asynchrones à l'aide d'**Axios**. Actuellement, chaque interaction utilisateur provoque un rechargement complet de la page, ce qui dégrade la fluidité de navigation.

Le projet comporte trois entités principales :

- **User** : identifié par un e-mail et un mot de passe
- **Article** : caractérisé par un titre, un contenu et une date de publication
- **Bookmark** : associe un utilisateur à un article (relation ManyToOne vers User et Article)

La base de données contient actuellement **12 utilisateurs**, **25 articles** et **40 bookmarks** générés via des fixtures Doctrine.

Architecture technique existante

Voici les éléments déjà en place dans le projet :

Élément	Détail
Framework	Symfony 5.4
Moteur de templates	Twig
Client HTTP JS	Axios (chargé via CDN)
Icônes	Font Awesome 5
CSS	Bootstrap 4
Authentification	Formulaire de connexion basique
Bundle Maker	Installé et opérationnel

Le contrôleur **ArticleController** expose actuellement une route racine `/` qui affiche tous les articles. Un contrôleur **UserController** expose une route `/users` renvoyant la liste des utilisateurs au format JSON grâce à une méthode `toArray()` sur l'entité User.

L'entité **Article** dispose déjà d'une méthode `isBookmarkedByUser(User $user): bool` qui parcourt la collection `$bookmarks` de l'article et renvoie `true` si l'utilisateur passé en paramètre a déjà mis l'article en favori.

Voici le contrôleur `BookmarkController` actuel, qui présente un **bug volontaire** :

```php /**

- @Route("/article/{id}/bookmark", name="article_bookmark")


Tous droits réservés à STUDI - Reproduction interdite
```

*/ public function toggleBookmark(Article $article, EntityManagerInterface $em, BookmarkRepository $bookmarkRepo): Response { $user = $this->getUser(); if(!$user) { return $this->json(["code" => 401, "message" => "Non authentifié"], 401); } if($article->isBookmarkedByUser($user)) { $bookmark = $bookmarkRepo->findOneBy(["article" => $article, "user" => $user]); $em->remove($bookmark); $em->flush(); return $this->json(["code" => 200, "message" => "Bookmark supprimé", "bookmarks" => $bookmarkRepo->count(["article" => $article])], 200); } $bookmark = (new Bookmark()) ->setArticle($article) ->setUser($user); $em->persist($bookmark); // $em->flush(); // <-- ligne commentée return $this->json(["code" => 200, "message" => "Bookmark ajouté", "bookmarks" => $bookmarkRepo->count(["article" => $article])], 200); } ```

Template Twig fourni

Voici le template `index.html.twig` que Samira vous demande de compléter et corriger :

```twig {% extends 'base.html.twig' %}

{% block body %} <h2>Articles NovaBlog</h2> <a href="/users" class="btn btn-primary js-users">Afficher les auteurs</a> <div class="users"></div> <div class="row"> {% for article in articles %} <div class="col-4 mb-3 card"> <div class="card-body"> <h5 class="card-title">{{ article.title }}</h5> <p class="card-text">{{ article.content|slice(0, 100) }}...</p> <a href="/article/{{ article.id }}/bookmark" class="btn btn-link js-bookmark"> {% if app.user and article.isBookmarkedByUser(app.user) %} <i class="fas fa-star"></i> {% else %} <i class="far fa-star"></i> {% endif %} <span class="js-count">{{ article.bookmarks|length }}</span> </a> </div> </div> {% endfor %} </div> {% endblock %}

{% block javascripts %} <script src="https://unpkg.com/axios/dist/axios.min.js"></script> <script> // TODO : implémenter la logique asynchrone pour les bookmarks // TODO : implémenter la logique asynchrone pour l'affichage des auteurs </script> {% endblock %} ```

### Question 1

Identifiez le bug présent dans la méthode toggleBookmark() du BookmarkController. Expliquez les conséquences de ce bug sur le comportement de l'application, tant côté serveur que côté client (réponse JSON renvoyée, état de la base de données, affichage utilisateur).

### Question 2

Rédigez le code JavaScript complet qui implémente la fonctionnalité de bookmark asynchrone dans le template Twig. Votre code doit gérer l'ajout et le retrait du favori au clic, la mise à jour du compteur et de l'icône, ainsi que les erreurs (utilisateur non authentifié et erreur générique). Justifiez le choix entre axios.get() et axios.post() pour cette fonctionnalité.

### Question 3

Samira vous demande d'implémenter également la fonctionnalité d'affichage asynchrone des auteurs (bouton « Afficher les auteurs »). Cependant, elle ajoute une contrainte : si la requête échoue avec un code 500, le bouton doit être désactivé visuellement (classe CSS `disabled` ajoutée, attribut `pointer-events: none`) et afficher le texte « Service indisponible ». Rédigez le code JavaScript correspondant et expliquez pourquoi vous utilisez axios.get() et non axios.post() dans ce cas.

### Question 4

Samira envisage de remplacer la méthode toArray() de l'entité User par l'utilisation du Serializer Symfony pour convertir les entités en JSON. Analysez les avantages et les limites de chacune des deux approches (méthode toArray() manuelle vs Serializer Symfony) en vous appuyant sur des critères de maintenabilité, de sécurité des données et de flexibilité. Formulez une recommandation argumentée pour le projet NovaBlog.

Tous droits réservés à STUDI - Reproduction interdite
```

B. Test

Exercice 1 : Principes fondamentaux de l'AJAX

Exercice

L'AJAX est un langage de programmation à part entière, distinct de JavaScript, permettant de réaliser des échanges asynchrones entre le client et le serveur.

- Vrai
- Faux

Exercice

Parmi les propositions suivantes, laquelle décrit le mieux ce que permet concrètement l'AJAX dans une application web ?

- Recharger entièrement la page à chaque interaction utilisateur pour garantir la cohérence des données affichées.
- Mettre à jour une partie du contenu d'une page web sans rechargement complet, grâce à des échanges asynchrones avec le serveur.
- Remplacer le serveur HTTP par un traitement entièrement côté client, sans aucune requête réseau.
- Générer automatiquement des templates Twig à partir de données JSON reçues du serveur.

Exercice

Quelles étapes font partie du cycle d'une interaction AJAX telle que décrite dans la leçon ? Sélectionnez toutes les réponses correctes.

- **A** L'utilisateur interagit avec un élément du DOM, comme un bouton.
- **B** Le navigateur recharge entièrement la page pour afficher les nouvelles données.
- **C** Le serveur traite la requête reçue depuis le client.
- **D** Le client reçoit la réponse traitée par le serveur.
- **E** Le serveur génère automatiquement le JavaScript nécessaire à la mise à jour du DOM.

Exercice 5 : Axios et requêtes asynchrones

Exercice

La méthode `axios.get()` retourne une promesse JavaScript, ce qui signifie que le code placé dans le bloc `then()` s'exécute immédiatement, sans attendre la réponse du serveur.

Tous droits réservés à STUDI - Reproduction interdite

Vrai

Faux

Exercice

Dans la fonctionnalité de 'like' sur un article, la leçon utilise `axios.post()` plutôt que `axios.get()`. Quelle est la raison principale de ce choix ?

La méthode `get()` ne permet pas de passer une URL en argument, contrairement à `post()`.

La méthode `post()` est utilisée car l'opération vise à créer ou modifier des données côté serveur, ce qui correspond sémantiquement au verbe HTTP POST.

La méthode `get()` ne supporte pas les promesses JavaScript, alors que `post()` les supporte nativement.

La méthode `post()` est obligatoire dès lors que la route Symfony contient un paramètre dynamique comme `{id}`.

Exercice

Concernant la gestion des erreurs avec Axios, quelles affirmations sont correctes d'après la leçon ? Sélectionnez toutes les réponses valides.

A ☐ La méthode `catch()` reçoit en argument une fonction anonyme qui prend l'erreur en paramètre.

B La méthode `catch()` doit être placée avant le bloc `then()` pour intercepter les erreurs.

C Il est possible d'accéder au code de statut HTTP de l'erreur via `error.response.status`.

D ☐ En l'absence de bloc `catch()`, Axios interrompt automatiquement l'exécution du script sans aucun effet visible.

E ☐ Le bloc `catch()` permet de traiter différemment les erreurs selon leur code de statut, par exemple afficher un message spécifique pour une erreur 403.

Exercice

Remettez dans le bon ordre les étapes d'un appel asynchrone avec Axios, depuis le clic de l'utilisateur jusqu'à la mise à jour du DOM.

Tous droits réservés à STUDI - Reproduction interdite

A Le serveur traite la requête et renvoie une réponse au format JSON.

B Le bloc `then()` reçoit l'objet `response` et met à jour le DOM en conséquence.

C `event.preventDefault()` est appelé pour bloquer le comportement par défaut du navigateur.

D `axios.get()` ou `axios.post()` est appelé avec l'URL cible en argument.

E L'utilisateur clique sur un élément HTML auquel un écouteur d'événement a été attaché.

Réponse : ___ ___ ___ ___ ___

Exercice

Associez chaque élément de la syntaxe Axios à son rôle dans le traitement asynchrone.

A axios.post(url, data) **B** .then(response => { ... }) **C** axios.get(url)

D .catch(error => { ... })

Envoie une requête HTTP GET vers l'URL spécifiée et retourne une promesse	Envoie une requête HTTP POST avec des données dans le corps de la requête	Exécute un traitement une fois la réponse HTTP reçue avec succès	Intercepte et traite les erreurs survenues lors de la requête asynchrone

Exercice 11 : Architecture Symfony pour les réponses JSON

Exercice

Dans le contrôleur `UserController`, la méthode `json()` est utilisée pour renvoyer les données au client. Quel est l'apport du second argument optionnel que cette méthode peut recevoir ?

Il permet de définir le format de sérialisation des données (XML, JSON, CSV).

Il permet de spécifier le code de statut HTTP associé à la réponse, par exemple 200 ou 403.

Il permet de passer des en-têtes HTTP personnalisés directement dans la réponse.

Il permet de définir le nom du fichier téléchargé si la réponse est de type binaire.

Exercice

Associez chaque composant ou méthode Symfony à son rôle dans la mise en place d'une API JSON asynchrone.

Tous droits réservés à STUDI - Reproduction interdite

A Méthode getUser() dans le contrôleur
B PostLikeRepository::findOneBy()
C EntityManagerInterface
D Méthode toArray() sur l'entité

Convertit un objet PHP en tableau associatif pour le sérialiser en JSON	Récupère l'instance de l'utilisateur actuellement connecté	Permet de persister, modifier ou supprimer des entités en base de données	Recherche un enregistrement en base selon des critères précis

Exercice

Dans la méthode `like()` du `PostController`, quelles actions sont réalisées lorsque l'utilisateur a **déjà** aimé l'article ? Sélectionnez toutes les réponses correctes.

- [ ] **A** On récupère l'instance de `PostLike` correspondante via `findOneBy()`.
- [ ] **B** On crée une nouvelle instance de `PostLike` et on la persiste en base.
- [ ] **C** On supprime le `PostLike` existant via `$em->remove()` suivi de `$em->flush()`.
- [ ] **D** On renvoie une réponse JSON avec un code de statut 200 et le nouveau nombre de likes.
- [ ] **E** On renvoie une réponse JSON avec un code de statut 403 pour signaler l'interdiction.

Tous droits réservés à STUDI - Reproduction interdite